DATABASE MANAGEMENT SYSTEMS

PROJECT REPORT

Blood Donation Management System

Submitted By:

Member	Role
NAVNEET KUMAR (2024UIC3514)	System Description & ER Model
MADHAV DHIMAN. (2024UIC3515)	Relational Schema & Normalization
TANISHQ JAIN. (2024UIC3516)	SQL Implementation & Queries

TASK 1: System Description

1.1 Overview

A Blood Donation Management System (BDMS) is a centralized digital platform designed to efficiently manage the entire process of blood donation — from donor registration and blood collection to storage, testing, and distribution to hospitals and patients in need. It bridges the gap between blood donors and recipients, ensuring timely and safe blood supply.

1.2 Key Functionalities

- Donor Registration & Management: Register and manage blood donor profiles including personal details, blood group, and donation history.
- Blood Collection & Testing: Record blood donation events, test results (HIV, Hepatitis, etc.), and ensure only safe blood is stored.
- Blood Bank Inventory: Track blood units by type (A+, A-, B+, B-, AB+, AB-, O+, O-), quantity, expiry date, and storage location.
- Hospital Blood Requests: Allow hospitals to place requests for specific blood types and manage fulfillment.

- Recipient Management: Maintain patient records and match recipients with appropriate blood donors/units.
- Notifications & Alerts: Trigger alerts for low blood stock, upcoming expiry of blood units, and urgent donation requests.
- Reports & Analytics: Generate reports on donation trends, blood availability, hospital usage, and donor statistics.

1.3 Stakeholders

Stakeholder	Role in System
Donor	Registers, donates blood, views donation history
Blood Bank Staff	Manages inventory, records donations, tests blood
Hospital	Places blood requests, receives blood units
Patient / Recipient	Receives blood from hospital after matching
Admin	Manages overall system, users, and generates reports

TASK 2: Entity-Relationship (ER) Model

2.1 Entities and Attributes

Entity 1: DONOR

Attribute	Data Type	Constraint	Description
donor_id	INT	PRIMARY KEY	Unique donor identifier
first_name	VARCHAR(50)	NOT NULL	Donor's first name
last_name	VARCHAR(50)	NOT NULL	Donor's last name
date_of_birth	DATE	NOT NULL	Date of birth
gender	CHAR(1)	NOT NULL	M/F/O
blood_group	VARCHAR(5)	NOT NULL	e.g., A+, O-
phone	VARCHAR(15)	UNIQUE	Contact number
email	VARCHAR(100)	UNIQUE	Email address
address	TEXT	-	Residential address
registration_date	DATE	NOT NULL	Date of registration

Entity 2: BLOOD_BANK

Attribute	Data Type	Constraint	Description
bank_id	INT	PRIMARY KEY	Unique blood bank ID
bank_name	VARCHAR(100)	NOT NULL	Name of blood bank
location	VARCHAR(200)	NOT NULL	City and address

contact_number	VARCHAR(15)	-	Contact phone
email	VARCHAR(100)	-	Bank email
capacity	INT	-	Max units storage capacity

Entity 3: BLOOD_INVENTORY

Attribute	Data Type	Constraint	Description
inventory_id	INT	PRIMARY KEY	Unique inventory record ID
bank_id	INT	FOREIGN KEY	References BLOOD_BANK
blood_group	VARCHAR(5)	NOT NULL	Blood type stored
quantity_units	INT	NOT NULL	Number of units available
collection_date	DATE	NOT NULL	Date blood was collected
expiry_date	DATE	NOT NULL	Expiry date of blood unit
status	VARCHAR(20)	NOT NULL	Available/Reserved/ Expired

Entity 4: DONATION

Attribute	Data Type	Constraint	Description
donation_id	INT	PRIMARY KEY	Unique donation ID
donor_id	INT	FOREIGN KEY	References DONOR
bank_id	INT	FOREIGN KEY	References BLOOD_BANK
donation_date	DATE	NOT NULL	Date of donation
quantity_ml	INT	NOT NULL	Volume donated in ml
hiv_test	VARCHAR(10)	NOT NULL	Positive/Negative
hepatitis_test	VARCHAR(10)	NOT NULL	Positive/Negative
malaria_test	VARCHAR(10)	NOT NULL	Positive/Negative
test_result	VARCHAR(10)	NOT NULL	Safe/Unsafe overall

Entity 5: HOSPITAL

Attribute	Data Type	Constraint	Description
hospital_id	INT	PRIMARY KEY	Unique hospital ID
hospital_name	VARCHAR(100)	NOT NULL	Name of hospital
location	VARCHAR(200)	NOT NULL	Hospital address
contact_number	VARCHAR(15)	-	Contact number
email	VARCHAR(100)	-	Hospital email

Entity 6: BLOOD_REQUEST

Attribute	Data Type	Constraint	Description
request_id	INT	PRIMARY KEY	Unique request ID
hospital_id	INT	FOREIGN KEY	References HOSPITAL

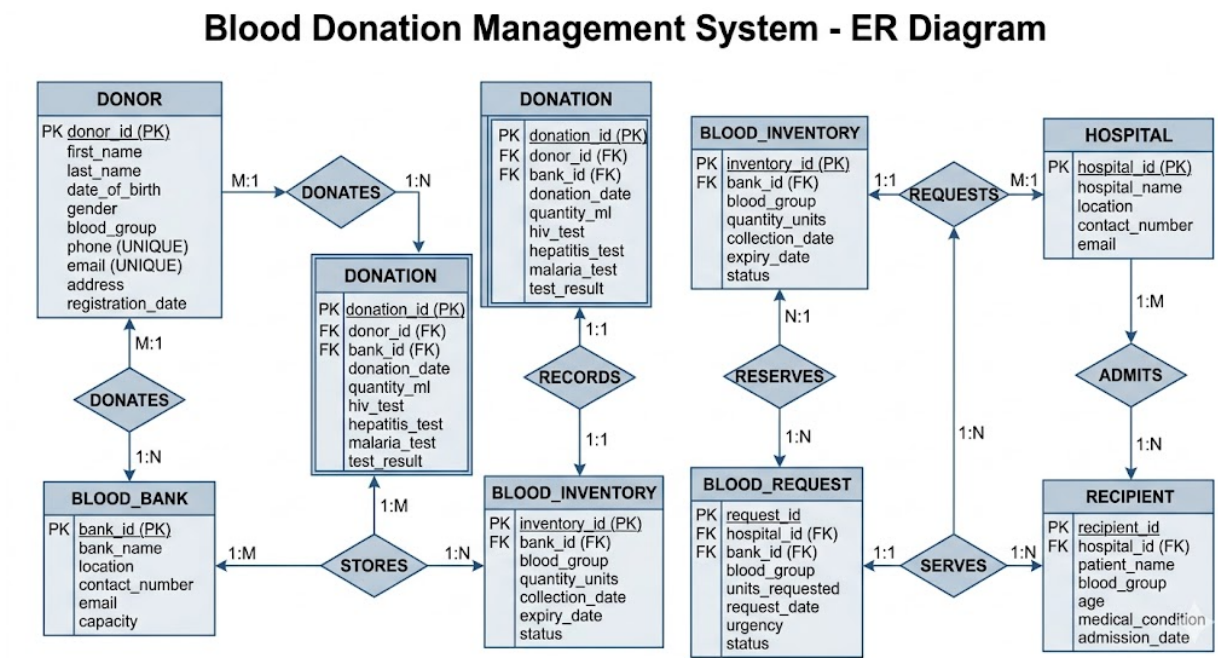
bank_id	INT	FOREIGN KEY	References BLOOD_BANK
blood_group	VARCHAR(5)	NOT NULL	Requested blood type
units_requested	INT	NOT NULL	Number of units needed
request_date	DATE	NOT NULL	Date of request
urgency	VARCHAR(10)	NOT NULL	Low/Normal/Critical
status	VARCHAR(15)	NOT NULL	Pending/Fulfilled/ Rejected

Entity 7: RECIPIENT

Attribute	Data Type	Constraint	Description
recipient_id	INT	PRIMARY KEY	Unique recipient ID
hospital_id	INT	FOREIGN KEY	References HOSPITAL
patient_name	VARCHAR(100)	NOT NULL	Patient full name
blood_group	VARCHAR(5)	NOT NULL	Patient blood group
age	INT	-	Patient age
medical_condition	TEXT	-	Reason for transfusion
admission_date	DATE	-	Date admitted

2.2 Relationships Summary

Relationship	Entity 1	Entity 2	Cardinality	Description
DONATES	DONOR	BLOOD_BANK	M : N (via DONATION)	A donor donates to one or more blood banks
STORES	BLOOD_BANK	BLOOD_INVENTORY	1 : M	A blood bank stores many blood inventory records
REQUESTS	HOSPITAL	BLOOD_BANK	M : N (via BLOOD_REQUEST)	A hospital requests blood from a blood bank
ADMITS	HOSPITAL	RECIPIENT	1 : M	A hospital admits many recipients/ patients
RECEIVES	RECIPIENT	BLOOD_REQUEST	1 : 1	A recipient is served by a blood request



TASK 3: Relational Schema

Each entity from the ER model is converted into a relation (table). Foreign keys are derived from relationships identified in the ER model.

3.1 Schema Definitions

DONOR

```
DONOR (donor_id PK, first_name, last_name, date_of_birth,
        gender, blood_group, phone UNIQUE, email UNIQUE,
        address, registration_date)
```

BLOOD_BANK

```
BLOOD_BANK (bank_id PK, bank_name, location,
             contact_number, email, capacity)
```

DONATION (Bridge: DONOR ↔ BLOOD_BANK)

```
DONATION (donation_id PK, donor_id FK → DONOR,
          bank_id FK → BLOOD_BANK, donation_date,
          quantity_ml, hiv_test, hepatitis_test,
          malaria_test, test_result)
```

BLOOD_INVENTORY

```
BLOOD_INVENTORY (inventory_id PK, bank_id FK → BLOOD_BANK,
                 blood_group, quantity_units, collection_date,
                 expiry_date, status)
```

HOSPITAL

```
HOSPITAL (hospital_id PK, hospital_name, location,
          contact_number, email)
```

BLOOD_REQUEST (Bridge: HOSPITAL ↔ BLOOD_BANK)

```
BLOOD_REQUEST (request_id PK, hospital_id FK → HOSPITAL,
               bank_id FK → BLOOD_BANK, blood_group,
               units_requested, request_date,
               urgency, status)
```

RECIPIENT

```
RECIPIENT (recipient_id PK, hospital_id FK → HOSPITAL,
           patient_name, blood_group, age,
           medical_condition, admission_date)
```

3.2 Primary & Foreign Key Summary

Table	Primary Key	Foreign Keys
DONOR	donor_id	None
BLOOD_BANK	bank_id	None
DONATION	donation_id	donor_id → DONOR, bank_id → BLOOD_BANK
BLOOD_INVENTORY	inventory_id	bank_id → BLOOD_BANK
HOSPITAL	hospital_id	None
BLOOD_REQUEST	request_id	hospital_id → HOSPITAL, bank_id → BLOOD_BANK
RECIPIENT	recipient_id	hospital_id → HOSPITAL

TASK 4: Normalization

Normalization is the process of organizing a database to reduce redundancy and improve data integrity. We apply 1NF, 2NF, and 3NF to each relation.

4.1 Normal Forms — Definitions

- 1NF (First Normal Form): All attributes are atomic (no repeating groups or multi-valued attributes); each record is unique.
- 2NF (Second Normal Form): In 1NF AND every non-key attribute is fully functionally dependent on the entire primary key (no partial dependencies). Relevant only when the primary key is composite.
- 3NF (Third Normal Form): In 2NF AND no transitive dependencies exist (non-key attributes do not depend on other non-key attributes).

4.2 Normalization Analysis Per Table

DONOR Table

All attributes are atomic. Primary key is donor_id (single attribute), so no partial dependencies are possible. There are no transitive dependencies — name, DOB, gender, blood group, contact info all depend directly on donor_id.

Highest Normal Form Achieved: 3NF

BLOOD_BANK Table

All attributes are atomic. Single-column primary key bank_id. No transitive dependencies exist — name, location, contact, email, capacity all directly depend on bank_id.

Highest Normal Form Achieved: 3NF

DONATION Table

Primary key is donation_id (single attribute). donor_id and bank_id are foreign keys, not determinants of other attributes. hiv_test, hepatitis_test, malaria_test are independent attributes. test_result is derived from individual test results — this could be a transitive dependency.

Resolution: test_result is kept as a stored computed field for query efficiency (or it can be computed via a VIEW). With this note, the table achieves 3NF.

Highest Normal Form Achieved: 3NF

BLOOD_INVENTORY Table

Primary key: inventory_id (single). All attributes — bank_id, blood_group, quantity_units, collection_date, expiry_date, status — directly depend on inventory_id. No partial or transitive dependencies.

Highest Normal Form Achieved: 3NF

HOSPITAL Table

Primary key: hospital_id. All attributes directly depend on hospital_id. No transitive dependencies.

Highest Normal Form Achieved: 3NF

BLOOD_REQUEST Table

Primary key: request_id. All non-key attributes (hospital_id, bank_id, blood_group, units_requested, request_date, urgency, status) directly depend on request_id. No partial or transitive dependency.

Highest Normal Form Achieved: 3NF

RECIPIENT Table

Primary key: recipient_id. hospital_id is a foreign key. All other attributes directly depend on recipient_id. No transitive dependency exists.

Highest Normal Form Achieved: 3NF

4.3 Normalization Summary

Table	1NF	2NF	3NF	Highest NF
DONOR	✓	✓	✓	3NF
BLOOD_BANK	✓	✓	✓	3NF
DONATION	✓	✓	✓	3NF
BLOOD_INVENTORY	✓	✓	✓	3NF
HOSPITAL	✓	✓	✓	3NF
BLOOD_REQUEST	✓	✓	✓	3NF
RECIPIENT	✓	✓	✓	3NF

TASK 5: SQL Implementation

5.1 CREATE TABLE Statements

Create DONOR Table

```
CREATE TABLE DONOR (
    donor_id         INT PRIMARY KEY AUTO_INCREMENT,
    first_name       VARCHAR(50)  NOT NULL,
    last_name        VARCHAR(50)  NOT NULL,
    date_of_birth    DATE         NOT NULL,
    gender           CHAR(1)      NOT NULL CHECK (gender IN ('M','F','O')),
    blood_group      VARCHAR(5)   NOT NULL,
    phone            VARCHAR(15)  UNIQUE,
    email            VARCHAR(100) UNIQUE,
    address          TEXT,
    registration_date DATE        NOT NULL
);
```

Create BLOOD_BANK Table

```
CREATE TABLE BLOOD_BANK (
    bank_id          INT PRIMARY KEY AUTO_INCREMENT,
    bank_name        VARCHAR(100) NOT NULL,
    location         VARCHAR(200) NOT NULL,
    contact_number   VARCHAR(15),
    email            VARCHAR(100),
    capacity         INT
);
```

Create DONATION Table

```
CREATE TABLE DONATION (
    donation_id      INT PRIMARY KEY AUTO_INCREMENT,
    donor_id         INT NOT NULL,
    bank_id          INT NOT NULL,
    donation_date    DATE NOT NULL,
    quantity_ml      INT NOT NULL,
    hiv_test         VARCHAR(10) NOT NULL,
    hepatitis_test   VARCHAR(10) NOT NULL,
    malaria_test     VARCHAR(10) NOT NULL,
    test_result      VARCHAR(10) NOT NULL,
    FOREIGN KEY (donor_id) REFERENCES DONOR(donor_id),
    FOREIGN KEY (bank_id)  REFERENCES BLOOD_BANK(bank_id)
);
```

Create BLOOD_INVENTORY Table

```
CREATE TABLE BLOOD_INVENTORY (
```

```
    inventory_id     INT PRIMARY KEY AUTO_INCREMENT,
    bank_id          INT NOT NULL,
    blood_group      VARCHAR(5)  NOT NULL,
    quantity_units   INT NOT NULL,
    collection_date  DATE NOT NULL,
    expiry_date      DATE NOT NULL,
    status           VARCHAR(20) NOT NULL
                     DEFAULT 'Available',
    FOREIGN KEY (bank_id) REFERENCES BLOOD_BANK(bank_id)
);
```

Create HOSPITAL Table

```
CREATE TABLE HOSPITAL (
    hospital_id      INT PRIMARY KEY AUTO_INCREMENT,
    hospital_name    VARCHAR(100) NOT NULL,
    location         VARCHAR(200) NOT NULL,
    contact_number   VARCHAR(15),
    email            VARCHAR(100)
);
```

Create BLOOD_REQUEST Table

```
CREATE TABLE BLOOD_REQUEST (
    request_id       INT PRIMARY KEY AUTO_INCREMENT,
    hospital_id      INT NOT NULL,
    bank_id          INT NOT NULL,
    blood_group      VARCHAR(5)  NOT NULL,
    units_requested  INT NOT NULL,
    request_date     DATE NOT NULL,
    urgency          VARCHAR(10) NOT NULL
                     CHECK (urgency IN ('Low','Normal','Critical')),
    status           VARCHAR(15) NOT NULL DEFAULT 'Pending',
    FOREIGN KEY (hospital_id) REFERENCES HOSPITAL(hospital_id),
    FOREIGN KEY (bank_id)     REFERENCES BLOOD_BANK(bank_id)
);
```

Create RECIPIENT Table

```
CREATE TABLE RECIPIENT (
    recipient_id     INT PRIMARY KEY AUTO_INCREMENT,
    hospital_id      INT NOT NULL,
    patient_name     VARCHAR(100) NOT NULL,
    blood_group      VARCHAR(5)   NOT NULL,
    age              INT,
    medical_condition TEXT,
    admission_date   DATE,
    FOREIGN KEY (hospital_id) REFERENCES HOSPITAL(hospital_id)
);
```

5.2 INSERT Sample Data

Insert Donors

```
INSERT INTO DONOR VALUES
(1,'Rahul','Sharma','1990-05-12','M','A+',
 '9876543210','rahul@gmail.com','Delhi','2023-01-10'),
(2,'Priya','Verma','1995-08-22','F','B+',
 '9123456780','priya@gmail.com','Mumbai','2023-02-15'),
(3,'Amit','Singh','1988-03-30','M','O+',
 '9012345678','amit@gmail.com','Bangalore','2023-03-20'),
(4,'Sneha','Rao','2000-11-05','F','AB-',
 '8901234567','sneha@gmail.com','Hyderabad','2023-04-25'),
(5,'Vikas','Gupta','1993-07-17','M','O-',
 '8812345678','vikas@gmail.com','Chennai','2023-05-01');
```

Insert Blood Banks

```
INSERT INTO BLOOD_BANK VALUES
(1,'LifeLine Blood Bank','Connaught Place, Delhi',
 '011-23456789','lifeline@bb.com',500),
(2,'RedCross Blood Center','Andheri, Mumbai',
 '022-34567890','redcross@bb.com',800),
(3,'HealWell Blood Bank','Indiranagar, Bangalore',
 '080-45678901','healwell@bb.com',400);
```

Insert Donations

```
INSERT INTO DONATION VALUES
(1,1,1,'2023-06-01',450,'Negative','Negative','Negative','Safe'),
(2,2,1,'2023-06-05',450,'Negative','Negative','Negative','Safe'),
(3,3,2,'2023-06-10',450,'Negative','Negative','Negative','Safe'),
(4,4,2,'2023-06-15',450,'Negative','Positive','Negative','Unsafe'),
(5,5,3,'2023-06-20',450,'Negative','Negative','Negative','Safe');
```

Insert Blood Inventory

```
INSERT INTO BLOOD_INVENTORY VALUES
(1,1,'A+',50,'2023-06-01','2023-09-01','Available'),
(2,1,'B+',30,'2023-06-05','2023-09-05','Available'),
(3,2,'O+',70,'2023-06-10','2023-09-10','Available'),
(4,2,'AB-',10,'2023-06-15','2023-09-15','Reserved'),
(5,3,'O-',25,'2023-06-20','2023-09-20','Available');
```

Insert Hospitals

```
INSERT INTO HOSPITAL VALUES
(1,'AIIMS Delhi','Ansari Nagar, New Delhi',
 '011-26588500','aiims@hospital.com'),
```

```
(2,'KEM Hospital','Parel, Mumbai',
 '022-24107000','kem@hospital.com'),
(3,'Fortis Hospital','Bannerghatta, Bangalore',
 '080-66214444','fortis@hospital.com');
```

Insert Blood Requests

```
INSERT INTO BLOOD_REQUEST VALUES
(1,1,1,'A+',10,'2023-07-01','Critical','Fulfilled'),
(2,2,2,'O+',15,'2023-07-03','Normal','Fulfilled'),
(3,3,3,'O-',5,'2023-07-05','Critical','Pending'),
(4,1,2,'B+',8,'2023-07-07','Low','Pending'),
(5,2,1,'AB-',3,'2023-07-10','Critical','Fulfilled');
```

Insert Recipients

```
INSERT INTO RECIPIENT VALUES
(1,1,'Mohan Das',    'A+', 45,'Surgery','2023-07-01'),
(2,2,'Kamla Nair',   'O+', 62,'Accident','2023-07-03'),
(3,3,'Raj Malhotra', 'O-', 33,'Anemia','2023-07-05'),
(4,1,'Sita Devi',    'B+', 55,'Cancer','2023-07-07'),
(5,2,'Arjun Kapoor', 'AB-',28,'Thalassemia','2023-07-10');
```

5.3 Data Retrieval Queries

Query 1: List all safe donations with donor names

Purpose: Retrieve all successful (safe) donations along with the donor's full name and blood group.

```
SELECT D.donor_id,
       CONCAT(D.first_name,' ',D.last_name) AS donor_name,
       D.blood_group,
       DN.donation_date,
       DN.quantity_ml,
       DN.test_result
FROM   DONOR D
JOIN   DONATION DN
ON D.donor_id = DN.donor_id
WHERE  DN.test_result = 'Safe'
ORDER BY DN.donation_date;
```

Query 2: Blood inventory status per blood bank

Purpose: Show total available blood units grouped by blood bank and blood group (aggregation).

```
SELECT BB.bank_name,
```

```
       BI.blood_group,
       SUM(BI.quantity_units) AS total_units,
       BI.status
FROM   BLOOD_BANK BB
JOIN   BLOOD_INVENTORY BI
ON BB.bank_id = BI.bank_id
GROUP BY BB.bank_name, BI.blood_group, BI.status
ORDER BY BB.bank_name, BI.blood_group;
```

Query 3: Critical blood requests with hospital and bank details

Purpose: Find all critical requests and show which hospital requested from which blood bank.

```
SELECT BR.request_id,
       H.hospital_name,
       BB.bank_name,
       BR.blood_group,
       BR.units_requested,
       BR.request_date,
       BR.status
FROM   BLOOD_REQUEST BR
JOIN   HOSPITAL H
ON BR.hospital_id = H.hospital_id
JOIN   BLOOD_BANK BB
ON BR.bank_id     = BB.bank_id
WHERE  BR.urgency = 'Critical'
ORDER BY BR.request_date;
```

Query 4: Count total donations per blood bank

Purpose: Aggregation query to find how many donations each blood bank received.

```
SELECT BB.bank_name,
       COUNT(DN.donation_id) AS total_donations,
       SUM(DN.quantity_ml)   AS total_ml_collected
FROM   BLOOD_BANK BB
LEFT JOIN DONATION DN ON BB.bank_id = DN.bank_id
GROUP BY BB.bank_name
ORDER BY total_donations DESC;
```

Query 5: Find donors who have never donated (new registrations)

Purpose: Identify newly registered donors who have not donated yet using a LEFT JOIN.

```
SELECT D.donor_id,
       CONCAT(D.first_name,' ',D.last_name) AS donor_name,
       D.blood_group,
       D.registration_date
FROM   DONOR D
LEFT JOIN DONATION DN
ON D.donor_id = DN.donor_id
WHERE  DN.donation_id IS NULL;
```

Query 6: Recipients matched with their hospital and blood group

Purpose: JOIN query showing all recipients, which hospital admitted them, and blood needed.

```
SELECT R.recipient_id,
       R.patient_name,
       R.blood_group,
       R.age,
       H.hospital_name,
       R.medical_condition,
       R.admission_date
FROM   RECIPIENT R
JOIN   HOSPITAL H
ON R.hospital_id = H.hospital_id
ORDER BY R.admission_date;
```

Query 7: Blood groups with low stock (less than 20 units)

Purpose: Alert query to find blood groups running critically low across all banks.

```
SELECT BB.bank_name,
       BI.blood_group,
       BI.quantity_units,
       BI.expiry_date
FROM   BLOOD_INVENTORY BI
JOIN   BLOOD_BANK BB
ON BI.bank_id = BB.bank_id
WHERE  BI.quantity_units < 20
  AND  BI.status = 'Available'
ORDER BY BI.quantity_units ASC;
```

Query 8: Most active donors (by number of donations)

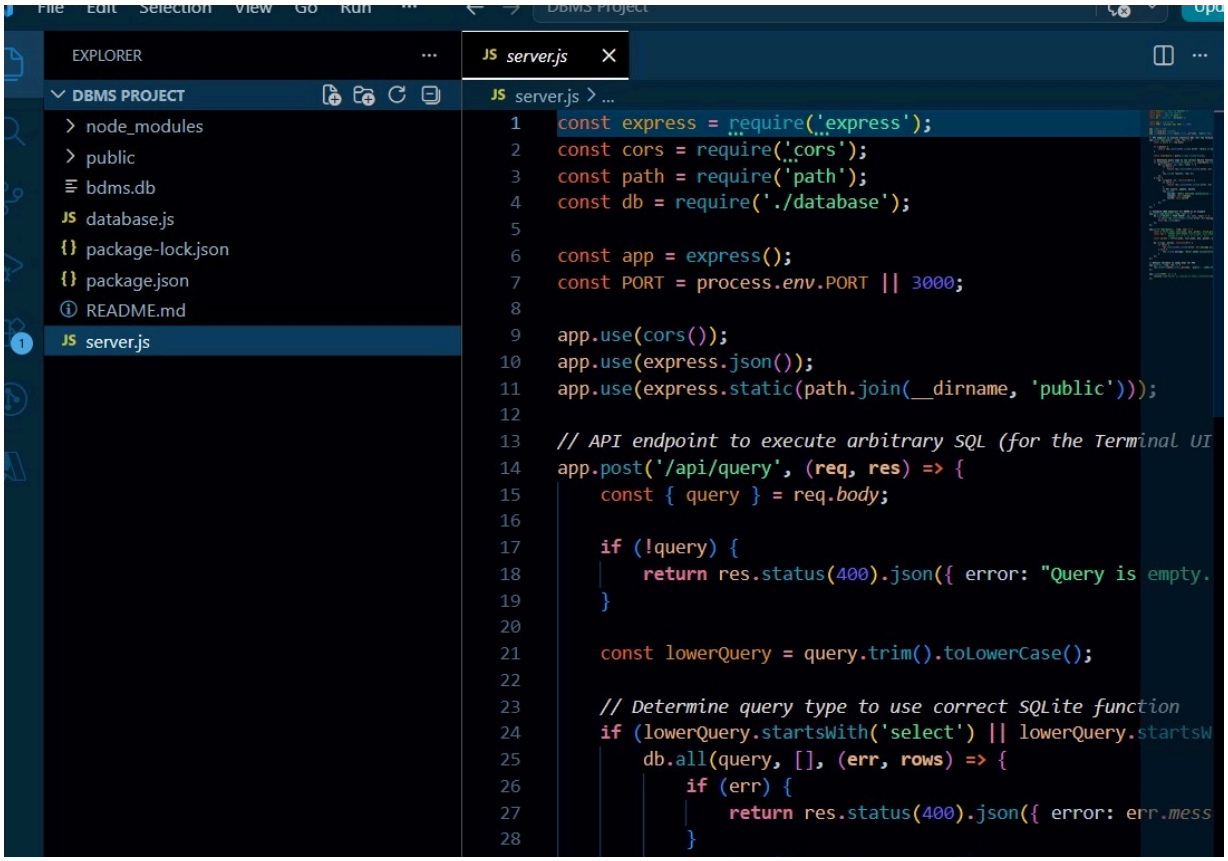
Purpose: Rank donors by total number of donations using GROUP BY and ORDER BY.

```
SELECT D.donor_id,
       CONCAT(D.first_name,' ',D.last_name) AS donor_name,
       D.blood_group,
       COUNT(DN.donation_id) AS times_donated
FROM   DONOR D
JOIN   DONATION DN ON D.donor_id = DN.donor_id
WHERE  DN.test_result = 'Safe'
GROUP BY D.donor_id, donor_name, D.blood_group
ORDER BY times_donated DESC;
```

FRONTEND

BDMS
Dashboard
Donors
SQL Terminal
Interactive SQL Terminal
Run queries directly against the database. DB Syncs in real-time.
sqlite> SELECT * FROM DONOR
donor_id | first_name | last_name | dob | gender | blood_group | phone | email
1 | John | Doe | 1990-05-15 | M | O+ | 1234567890 | john@example.com
2 | Jane | Smith | 1985-08-22 | F | A- | 0987654321 | jane@example.com
3 | Maddy | Boss | 2015-08-27 | M | O- | 9876543210 | maddygamer@gmail.com
4 | Aryan | Panwar the best CR | 2026-04-17 | F | A | 8447064121 | madhavdhiman.2503@g
sqlite> Enter SQL query here...

BACKEND



Conclusion

This project successfully demonstrates the design and implementation of a Blood Donation Management System using SQL and relational database principles. The key outcomes are:

- A comprehensive ER model was designed with 7 well-defined entities covering all major aspects of blood donation management.
- The relational schema clearly defines primary and foreign key constraints to maintain data integrity across all tables.
- All 7 tables were successfully normalized to Third Normal Form (3NF), eliminating redundancy and transitive dependencies.
- SQL DDL (CREATE TABLE) and DML (INSERT) statements were written to build and populate the database with realistic sample data.
- 8 meaningful SQL queries were written demonstrating JOINs (INNER, LEFT), GROUP BY, aggregations (COUNT, SUM), WHERE filters, and ORDER BY.

The Blood Donation Management System can be extended in future with features like automated expiry alerts, donor notification SMS integration, and a web-based front end for real-

time blood availability lookup.

— End of Report —